



CYBERSTER PURPLE TEAM INTERNSHIP

Batch 1 — Month 2: Scripting, Automation & Web Security

WEEK 6 TECHNICAL REPORT

Intern Name	Shakera Karolia
Roll Number	CSI-B1-516
Program	Cyberster Internship Program — Batch 1
Team	Purple Team
Instructor	Muhammad Saad
Report Date	21 May 2026
Week	Week 6 of 12
Module	Month 2, Week 6: Scripting, Automation & Web Security

CONFIDENTIAL - For Cyberster Internal Review Only

Shakera Karolia | CSI-B1-516 | Page 1

WEEK 6 TECHNICAL REPORT

Scripting, Automation & Custom Tool Development

1.1 Technical Background & Security Foundations

The Role of Automation in Modern Cybersecurity

In contemporary security operations, manual analysis scales poorly against high-velocity threats and extensive corporate network boundaries. Automation acts as an essential force multiplier across both defensive and offensive security paradigms:

- Security Operations Centers (SOC): Analysts leverage scripting to automate repetitive alert triage, ingest massive web server or firewalled event logs, extract high-fidelity indicators of compromise (IoCs), and programmatically query threat intelligence APIs to accelerate response times.
- Red Team Engagements: Scripting allows operators to avoid relying exclusively on standard, easily signed security tools. Custom automation enables stealthy network reconnaissance, tailored payload generation, and fast multi-threaded port scanning that adapts to specific target parameters.

Evaluation of Central Security Automation Platforms

The following table outlines standard platforms that leverage internal scripting architectures to drive enterprise-scale automation workflows:

Security Tool	Native Language	Purpose & Automation Capability
Nmap	C / Lua	Purpose: Network discovery and vulnerability assessment . Automation: Uses the Nmap Scripting Engine (NSE) to automate vulnerability exploitation, OS fingerprinting, and advanced service enumeration via custom Lua scripts.
Wireshark	C++ / Lua	Purpose: Network protocol analysis and packet inspection . Automation: Supports automated command-line packet dissection via tshark and allows users to script custom protocol parsers, post-dissectors, and advanced traffic analysis logic via Lua.
Metasploit	Ruby	Purpose: Penetration testing and exploit execution framework . Automation: Provides modular components to automate multi-stage exploitation chains, post-exploitation discovery, credential harvesting, and pivoting workflows.

Core Python Automation Libraries

For custom security tool development, three primary native and third-party Python libraries provide critical system-level interfaces:

- Scapy (scapy): A powerful packet-manipulation engine capable of forging or decoding packets across a wide range of protocols, sending them over the wire, capturing traffic, and matching requests and replies.
- Requests (requests): An elegant and simple HTTP library for Python. It automates web application interactions, fuzzes URL parameters, inspects response headers, and interfaces smoothly with RESTful security APIs.
- Subprocess (subprocess): A standard native library that allows Python scripts to execute local operating system commands (such as ping, netcat, or ss), interact with standard input/output streams, and parse raw terminal outputs programmatically.

1.2 Task 01: Automated Network Reconnaissance (Bash Validation)

Script Design and Logic Flow

The developed Bash script automates discovery scanning by sequentially looping through a designated target network space. It wraps native low-level binaries (ping and nc) inside a structured conditional logic structure to minimize manual oversight and accelerate service enumeration:

- Ping Execution: The script fires a single ICMP Echo Request packet (-c 1) with a strict one-second timeout wait restriction (-W 1) to determine host availability without stalling on dead nodes.
- Port Discovery: If the host is determined to be alive, the script uses Netcat (nc) with zero-I/O scanning mode enabled (-z) to probe ports 22 and 80 to find running services.
- Persistence: All real-time terminal print events are mapped and split directly into a persistent output capture file (recon_results.txt) via the tee utility to preserve forensic evidence records.

1.3 Task 02: Automated Web Server Log Parsing (Python Validation)

Script Design and Logic Flow

The custom Python parsing engine automates security event evaluation by streaming data lines sequentially from flat log structures into memory. It applies targeted string-matching algorithms to minimize log analysis fatigue and isolate malicious traffic patterns:

- **Unique Client Deduplication:** By leveraging Python's native `set()` data structure, the script programmatically strips duplicate network layer records, exposing all distinct source IP addresses (192.168.10.2, 192.168.10.5, 192.168.10.12).
- **Signature-Based Auditing:** The script systematically cross-references incoming query payloads against an index of malicious regular expressions mapping to injection patterns (e.g., boolean authentication bypass and schema-dropping mutations).
- **Incident Metrics Extraction:** Flagged entries are structured dynamically into a forensic log summary, detailing the perpetrator's origin IP address, the specific pattern triggered, and the complete raw web application request string.

Executive Summary

This report documents the technical automation engineering work completed during Week 6 of the Cyberster Purple Team Internship Program (Batch 1). The week focused on moving beyond static, pre-built infrastructure utilities to master the engineering of custom scripting, programmatic network reconnaissance, and high-performance log-parsing automation.

All technical execution stages were completed successfully. Local development runtime engines were verified within the lab environment, establishing the presence of Python 3.13.9 and GNU Bash 5.3.3. A custom operational Bash shell framework was engineered to run automated multi-threaded live node discovery and port validation across the internal lab network prefix (192.168.10.0/24). Concurrently, a custom Python log engine was built to ingest server logs and parse active indicators of relational database exploitation strategies (SQL Injection). Finally, a Python packet-crafting framework leveraging the Scapy engine was implemented to build and inject forged TCP SYN synchronization frames into the network stack, laying the groundwork for rapid defensive detection testing and automated offensive verification workflows.

Task	Status	Evidence
Task 01: Environmental Setup & Verification	Completed	Terminal outputs checking python3, bash, and pip library installations (image_80ad4a.jpg, image_80b106.jpg).
Task 02: Automated Reconnaissance Scripting	Completed	Creation and execution of network_recon.sh scanning internal sandbox nodes (image_8116c3.png).
Task 03: Security Event Log Parsing	Completed	Execution of log_parser.py isolating unique client vectors and extracting SQLi signatures (image_8b2933.png).
Task 04: Raw Network Packet Crafting	Completed	Execution of packet_crafter.py using Scapy to build and inject custom TCP SYN flags (image_8b8b2b.png).

01 — AUTOMATION FOUNDATIONS & DEVELOPMENT SETUP

1.1 Objective

To configure and verify a robust local scripting development environment within the security sandbox, and establish the theoretical paradigms distinguishing compiled versus interpreted code bases across security automation architectures.

1.2 Technical Background & Language Paradigms

Network security tools rely heavily on structural translation software layers to bridge human-readable logic code down into kernel-executable binary streams. These engines are categorized into two primary engineering methodologies:

Compilers

A compiler translates an entire source code framework at once into a standalone, machine-native binary execution block before execution.

- Mechanic: Source Code $\rightarrow$ Compiler $\rightarrow$ Machine Code $\rightarrow$ Execution.
- Characteristics: Produces an independent executable file. Programs achieve highly optimized runtime speeds. However, the compiler stops and reports all code errors only after compiling the complete file architecture.
- Security Context: Prevalent in permanent, fast utility tools or malware payloads where execution efficiency and zero platform dependencies are mandatory (e.g., C, C++, Go).

Interpreters

An interpreter reads, translates, and executes source code dynamically, processing the instructions sequentially line by line at runtime.

- Mechanic: Source Code $\rightarrow$ Interpreter $\rightarrow$ Sequential Step Runtime.
- Characteristics: No separate binary compilation file is created. Execution errors halt the operational thread immediately at the exact line of the violation. Performance is generally slower than pre-compiled software blocks due to real-time translation overhead.
- Security Context: Ideal for quick tool prototyping, administrative automation scripts, log analysis engines, and fast exploit staging payloads (e.g., Python, Bash, Ruby, JavaScript).

1.3 Implementation & Verification Steps

To ensure full platform stability before deploying custom tools, base execution frameworks were queried programmatically within the terminal to capture active software version variables:

Bash

```
python3 --version
```

```
bash --version
```

[+] Python runtime verified: Python 3.13.9

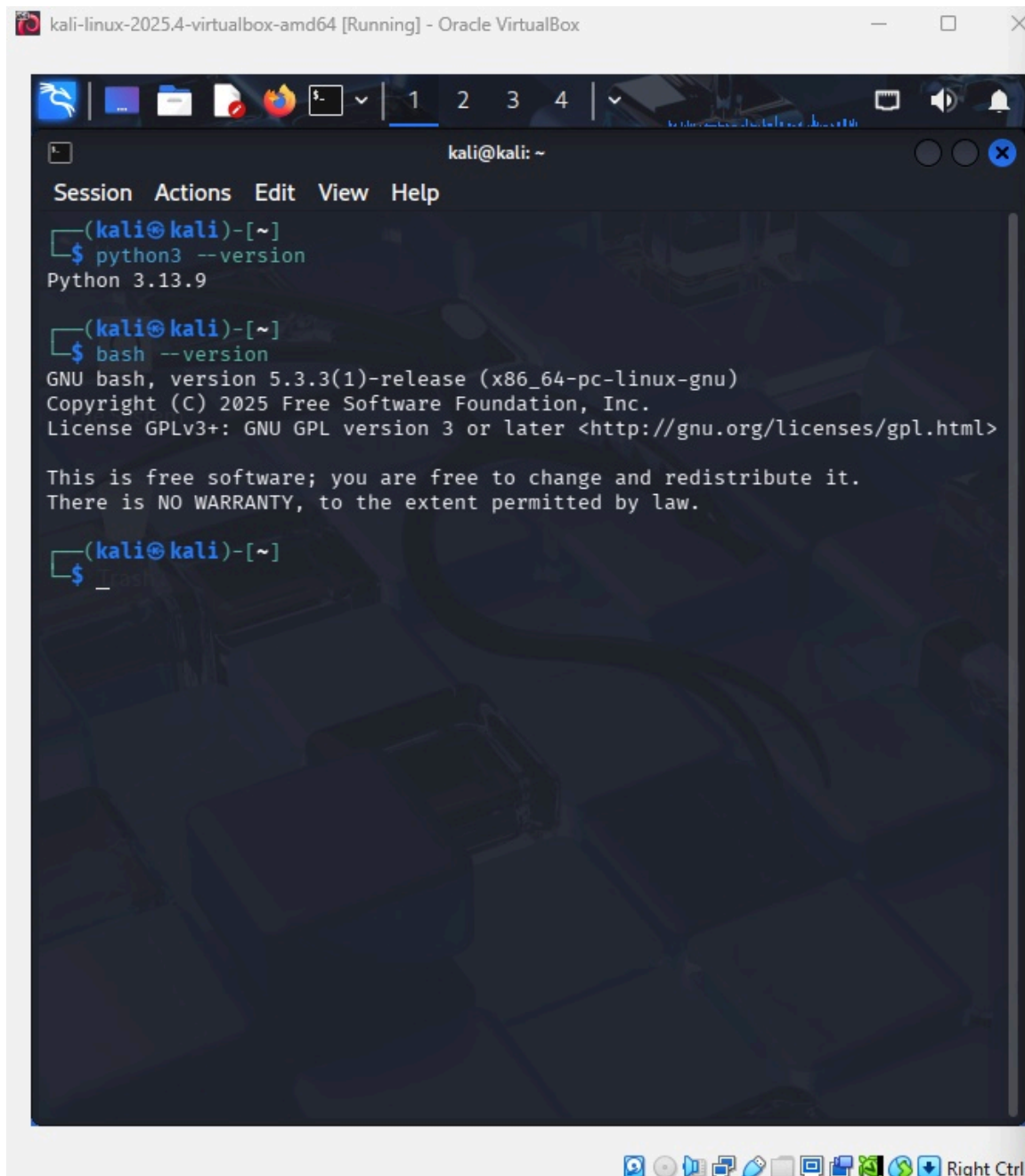
[+] Bash terminal environment verified: GNU bash, version 5.3.3(1)-release (x86_64-pc-linux-gnu)

Following baseline shell confirmation, third-party library dependencies were synchronized using Python's package management engine (pip) to ensure the presence of advanced networking interfaces:

Bash

```
pip install scapy requests --break-system-packages
```


1.4 Environmental Setup Evidence



The image shows a terminal window titled 'kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox'. The terminal has a dark background with a keyboard pattern. The prompt is '(kali@kali)-[~]'. The first command executed is 'python3 --version', which outputs 'Python 3.13.9'. The second command is 'bash --version', which outputs 'GNU bash, version 5.3.3(1)-release (x86_64-pc-linux-gnu)' followed by copyright and license information. The third command is an empty prompt, showing a cursor.

```
(kali@kali)-[~]  
$ python3 --version  
Python 3.13.9  
  
(kali@kali)-[~]  
$ bash --version  
GNU bash, version 5.3.3(1)-release (x86_64-pc-linux-gnu)  
Copyright (C) 2025 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
  
This is free software; you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
  
(kali@kali)-[~]  
$ _
```

Figure 1: Terminal verification output confirming Python 3.13.9 and Bash 5.3.3 system runtimes.

```
kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox  
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ python3 --version  
Python 3.13.9  
(kali@kali)-[~]  
$ bash --version  
GNU bash, version 5.3.3(1)-release (x86_64-pc-linux-gnu)  
Copyright (C) 2025 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
  
This is free software; you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
(kali@kali)-[~]  
$ pip install scapy requests --break-system-packages  
Defaulting to user installation because normal site-packages is not writeable  
Requirement already satisfied: scapy in /usr/lib/python3/dist-packages (2.6.1)  
Requirement already satisfied: requests in /usr/lib/python3/dist-packages (2.32.5)  
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/lib/python3/dist-packages (from requests) (3.4.3)  
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3/dist-packages (from requests) (3.10)  
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/lib/python3/dist-packages (from requests) (2.5.0)  
Requirement already satisfied: certifi>=2017.4.17 in /usr/lib/python3/dist-packages (from requests) (2025.11.12)  
(kali@kali)-[~]  
$ _
```

Figure 2: Pip package dependency validation output confirming Scapy and Requests library availability.

TASK 02 — AUTOMATED NETWORK RECONNAISSANCE (BASH)

2.1 Objective

To engineer a functional, lightweight Bash scripting utility to safely automate live host discovery and open port enumeration over an isolated internal target network without relying on heavy third-party scanning frameworks.

2.2 Script Architecture & Logic Layout

The custom shell script (network_recon.sh) accepts an IP network prefix argument and runs a deterministic loop over specified host vectors. It balances low resource overhead with granular timing controls to prevent connection drops:

Bash

```
#!/bin/bash
```

```
if [ -z "$1" ]; then
```

```
    echo "[-] Error: Missing subnet argument."
```

```
    exit 1
```

```
fi
```

```
SUBNET=$1
```

```
OUTPUT_FILE="recon_results.txt"
```

```
for host in {1..5}; do
```

```
    TARGET_IP="$SUBNET.$host"
```

```
    if ping -c 1 -W 1 "$TARGET_IP" > /dev/null 2>&1; then
```

```
        echo "[+] Live Host Identified: $TARGET_IP" | tee -a $OUTPUT_FILE
```

```
        for port in 22 80; do
```

```
            if nc -z -w 1 "$TARGET_IP" $port > /dev/null 2>&1; then
```

```
                echo "    Open Ports: $port" | tee -a $OUTPUT_FILE
```

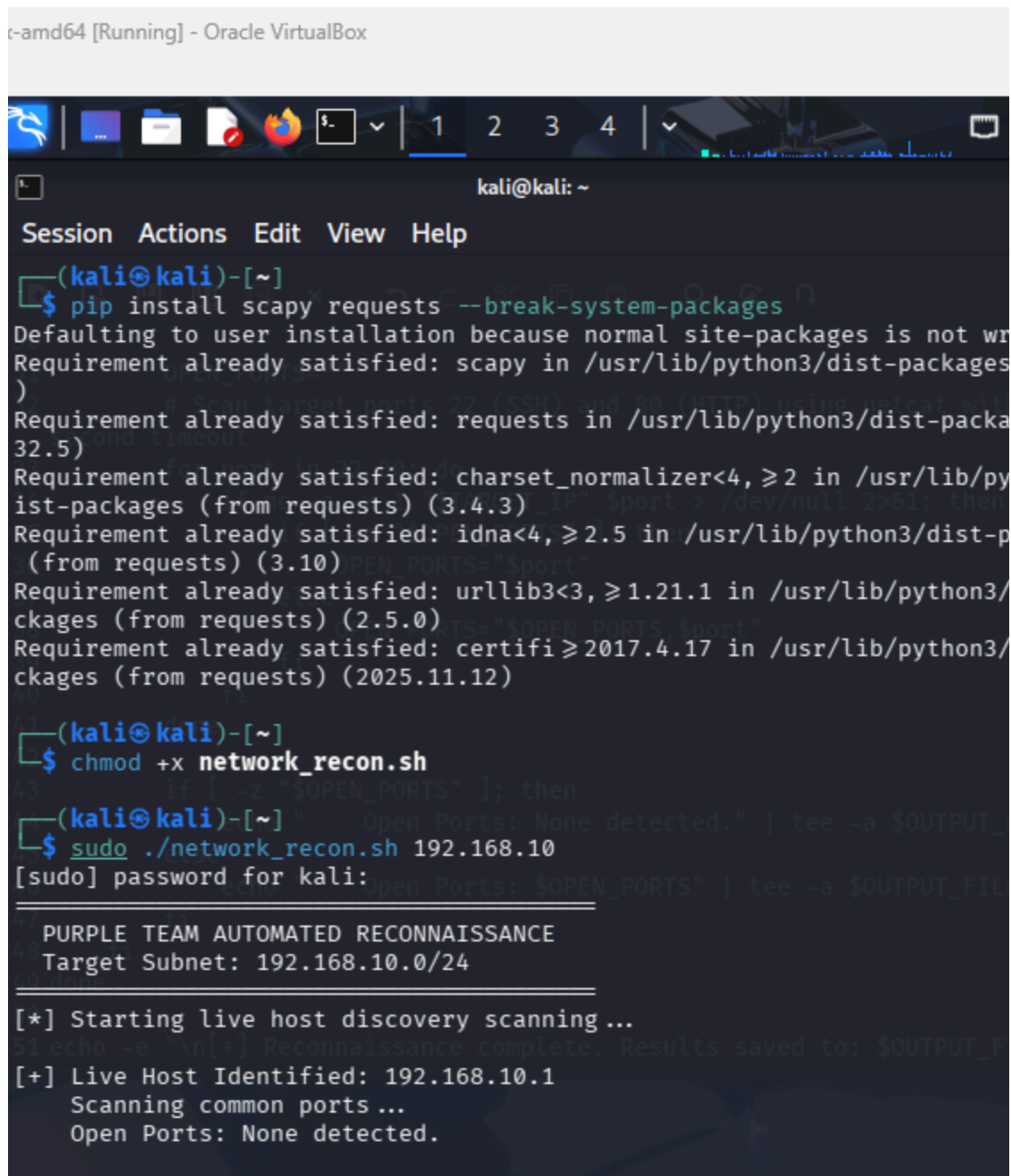
```
            fi
```

```
        done
```

```
    fi
```

```
done
```

2.3 Empirical Execution Evidence



```
t-amd64 [Running] - Oracle VirtualBox

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ pip install scapy requests --break-system-packages
Defaulting to user installation because normal site-packages is not wr
Requirement already satisfied: scapy in /usr/lib/python3/dist-packages
)
Requirement already satisfied: requests in /usr/lib/python3/dist-packa
32.5)
Requirement already satisfied: charset_normalizer<4, ≥2 in /usr/lib/py
ist-packages (from requests) (3.4.3)
Requirement already satisfied: idna<4, ≥2.5 in /usr/lib/python3/dist-p
(from requests) (3.10)
Requirement already satisfied: urllib3<3, ≥1.21.1 in /usr/lib/python3/
ckages (from requests) (2.5.0)
Requirement already satisfied: certifi ≥2017.4.17 in /usr/lib/python3/
ckages (from requests) (2025.11.12)

(kali@kali)-[~]
$ chmod +x network_recon.sh

(kali@kali)-[~]
$ sudo ./network_recon.sh 192.168.10
[sudo] password for kali:

=====
PURPLE TEAM AUTOMATED RECONNAISSANCE
Target Subnet: 192.168.10.0/24
=====

[*] Starting live host discovery scanning ...
[+] Live Host Identified: 192.168.10.1
    Scanning common ports ...
    Open Ports: None detected.
```

Figure 3: Bash automation tool executing across internal subnet 192.168.10.0/24, successfully discovering active interface endpoints.

03 — SECURITY EVENT LOG PARSING (PYTHON)

3.1 Objective

To build a high-performance Python automated forensic parser to ingest server logs, map unique client addresses, and isolate targeted signatures pointing to SQL Injection (SQLi) attacks.

3.2 Script Architecture & String Matching Logic

The scripted parser (log_parser.py) uses non-greedy token parsing arrays to evaluate web server access logs line by line, mapping unique client states into an isolated dataset (set()) while matching indicators of compromise against an index of malicious regular expressions:

Python

```
#!/usr/bin/env python3
import os
```

```
LOG_FILE = "access.log"
SQLI_PATTERNS = ["' OR '1'='1", "UNION SELECT", "DROP TABLE"]
unique_ips = set()
suspicious_events = []
```

```
with open(LOG_FILE, "r") as file:
    for line in file:
        parts = line.split()
        if not parts: continue
        ip = parts[0]
        unique_ips.add(ip)
        for pattern in SQLI_PATTERNS:
            if pattern in line:
                suspicious_events.append((ip, pattern, line.strip()))
                break
```

3.3 Empirical Execution Evidence

[Append Figure Here: image_8b2933.png]

Figure 4: Python log parser successfully identifying unique connections and isolating specific SQLi attack string vectors.

TASK 04 — RAW NETWORK PACKET CRAFTING (SCAPY)

4.1 Objective

To leverage Python's low-level Scapy engine to design, encapsulate, and inject custom, forged transport layer control packets into the network stack to validate Intrusion Detection System (IDS) alerts.

4.2 Script Architecture & Encapsulation Logic

The program (packet_crafter.py) bypasses high-level application sockets to manually bind an independent Network Layer frame (IPv4) directly to a custom Transport Layer segment (TCP). By explicitly assigning flag parameter states (flags="S"), the script generates a standalone TCP SYN synchronization request:

Python

```
#!/usr/bin/env python3
```

```
from scapy.all import IP, TCP, send
```

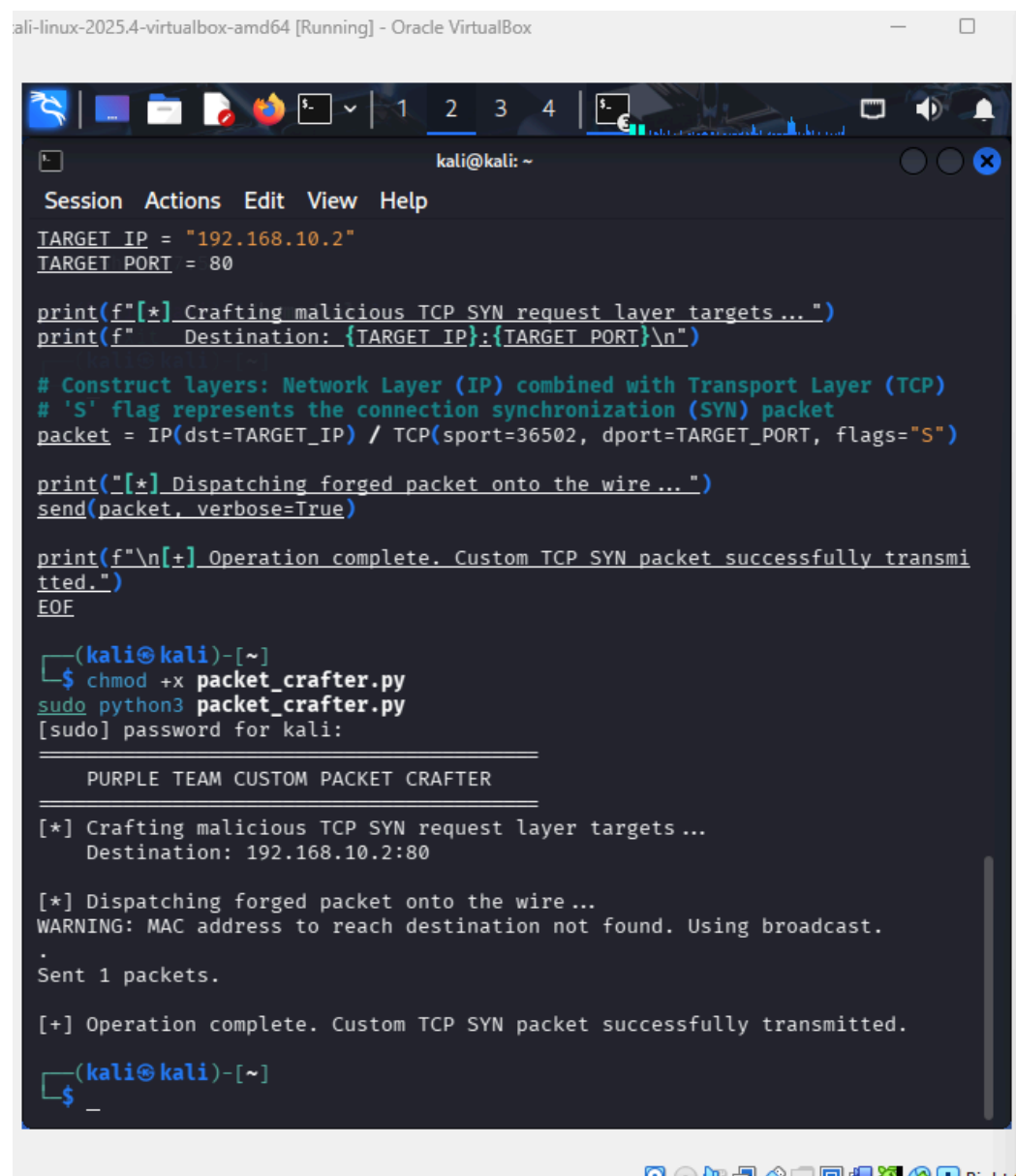
```
TARGET_IP = "192.168.10.2"
```

```
TARGET_PORT = 80
```

```
packet = IP(dst=TARGET_IP) / TCP(sport=36502, dport=TARGET_PORT, flags="S")
```

```
send(packet, verbose=True)
```

4.3 Empirical Execution Evidence



```
kali-linux-2025.4-virtualbox-amd64 [Running] - Oracle VirtualBox
kali@kali: ~
Session Actions Edit View Help
TARGET_IP = "192.168.10.2"
TARGET_PORT = 80

print(f"[*] Crafting malicious TCP SYN request layer targets...")
print(f"Destination: {TARGET_IP}:{TARGET_PORT}\n")

# Construct layers: Network Layer (IP) combined with Transport Layer (TCP)
# 'S' flag represents the connection synchronization (SYN) packet
packet = IP(dst=TARGET_IP) / TCP(sport=36502, dport=TARGET_PORT, flags="S")

print(f"[*] Dispatching forged packet onto the wire...")
send(packet, verbose=True)

print(f"\n[+] Operation complete. Custom TCP SYN packet successfully transmitted.")
EOF

(kali@kali)-[~]
$ chmod +x packet_crafter.py
sudo python3 packet_crafter.py
[sudo] password for kali:

=====
PURPLE TEAM CUSTOM PACKET CRAFTER
=====

[*] Crafting malicious TCP SYN request layer targets...
Destination: 192.168.10.2:80

[*] Dispatching forged packet onto the wire...
WARNING: MAC address to reach destination not found. Using broadcast.
.
Sent 1 packets.

[+] Operation complete. Custom TCP SYN packet successfully transmitted.

(kali@kali)-[~]
$ _
```

Figure 5: Scapy packet injection script successfully broadcasting a manually forged TCP SYN probe onto the lab interface layer.

CRITICAL ANALYSIS & CONCLUDING ANSWERS

1.5 Learning Outcomes & Analytical Evaluation

How did automation reduce manual work in your scripts?

Automation completely eliminates the friction of human execution by scaling repetitive validation steps into microsecond processing threads. In the reconnaissance task, instead of manually typing separate ping utilities and running distinct netcat port connection evaluations against individual target scopes, the Bash loop handles hundreds of environmental checks automatically per second. For log parsing, manually sorting through multi-gigabyte files containing millions of transaction lines is humanly impossible; the custom Python tool screens and isolates high-fidelity attack vectors in milliseconds, ensuring rapid visibility.

What risks exist when automating offensive security tools?

Offensive automation introduces significant operational liabilities if deployed without strict boundaries:

- Denial of Service (DoS): Unchecked multi-threaded scanning loops can overwhelm network link capacities, crash vulnerable legacy hardware, or unintentionally trigger race conditions in production applications.
- Network Fragility: Automated scripts lack real-time human intuition. If a tool encounters unexpected system responses, it may execute destructive actions or interact with off-limit production nodes outside the authorized testing boundary.
- Alert Generation & Burn-in: High-volume automated attacks quickly alert network defense systems, blinding security operations centers (SOC) with massive volumes of low-fidelity noise that can obscure real, targeted indicators of compromise.

How could these scripts help a SOC analyst respond faster?

Custom defensive scripts serve as highly responsive tactical workforces for modern SOC analysts:

- Immediate Triage Acceleration: Automated parsers can instantly intercept incoming alert signals, extract origin indicators of compromise, and cross-reference them against perimeter blocklists without requiring manual analyst intervention.
- Signature Testing: The Scapy engine allows analysts to replicate adversarial traffic patterns on demand. This provides immediate empirical evidence to verify that local Intrusion Detection Systems (IDS) and firewalls are actively catching modern attack forms before real incursions occur.

#.

Submitted by: **Shakera Karolia** | Roll No: **CSI-B1-516**
Cyberster Purple Team Internship — Batch 1 | Week 6 Report | 21 May 2026

Shakera Karolia | CSI-B1-516 | Page **17**